Los siguientes ejercicios son de iniciación a los punteros, pero tienen partes que obligan a una comprensión profunda de los conceptos asociados por lo que, aunque hayas manejado punteros antes en Pascal, es MUY RECOMENDABLE resolverlos.

Lo ideal es hacerlo en Lazarus (o tu entorno de desarrollo favorito de Pascal) donde irás escribiendo el código, y a la vez en un papel, donde irás haciendo los dibujos de los punteros, etc. No pienses que hacer dibujos no te aporta nada y puedes obviarlo, de verdad ayuda mucho pues la representación gráfica permite que trabaje tu memoria visual y los conceptos se fijen mejor. En serio: hazlo de las dos maneras.

1. **Sigue la siguiente secuencia de ejecución escribiendo código Pascal y a la vez dibuja en un papel lo que ocurre tras ejecutar cada instrucción [Nivel básico]:**

    1. Declara variable entera (x)
    2. Declara una variable de tipo puntero a entero (p_ent)
    3. Da el valor 100 a x
    4. Crea un entero dinámicamente con p_ent y dale el valor que tiene actualmente x
    5. Imprime por pantalla el valor contenido en el entero al que apunta p_ent
    6. Crea con new un nuevo entero dinámicamente
    7. Imprime por pantalla el valor contenido en el entero al que apunta p_ent
    8. Pon el puntero p_ent a apuntar a x
    9. Súmale 100 al entero apuntado por el puntero p_ent
    10. Imprime por pantalla el valor de x y también del entero al que apunta p_ent
    11. Pon el puntero p_ent a NIL
    12. Libera la memoria asociada al nuevo entero
    13. Suma 100 a x pero sin usar x en la operación de suma (solo usando p_ent)
    14. Muestra por pantalla que son iguales
    15. Libera toda la memoria asociada a p_ent y termina

    HAY DOS PUNTOS CONFLICTIVOS: Coméntalos.

**SOLUCIÓN** (entre paréntesis el número de instrucción a que da solución)**:**

```
program Ejercicios_punteros_01;
uses crt;
var
  x, i: integer; { (1) }
  p_ent: ^integer; { (2) }
begin
  x := 100; { (3) }
  new(p_ent); { (4) }
  p_ent^ := x; { (4) }
  writeln(p_ent^); { (5) }
  new(p_ent); { (6) Se pierde la referencia al primer entero creado }
  writeln(p_ent^); { (7) }
  p_ent := @x; { (8) }
  p_ent^:= p_ent^+ 100; { (9) }
  writeln('x=',x, 'p_ent=', p_ent^); { (10) }
```

```
    p_ent := NIL; { (11) }
    dispose(p_ent); { (12) No hace nada, pues no hay memoria dinámica
                           apuntada por p_ent, ya que p_ent apunta a un
                           lugar "seguro" (NIL) }
    p_ent := @x; { (13) }
    p_ent^:= p_ent^ + 100; { (13) }
    writeln('x=',x, 'p_ent=', p_ent^); { (14) }
    dispose(p_ent); { (15) Provoca un error pues no se puede liberar
                           memoria que no haya sido asignada
                          dinámicamente (heap): x es memoria de la pila}
  end.
```

2. **Sigue la siguiente secuencia de ejecución escribiendo código Pascal y a la vez dibuja en un papel lo que ocurre tras ejecutar cada instrucción [Nivel básico]**:

   1. Crea un array de 3 enteros V e inicialízalo con números aleatorios.
   2. Muestra sus valores por pantalla
   3. Declara un puntero a entero p y ponlo a apuntar a la primera posición del array
   4. Pon el valor 100 en el entero apuntado por p
   5. Muestra los valores del array "V" por pantalla
   6. Recorre con p todas las posiciones del array para ponerlas todas a cero

   *(Nota: puedes sumar 1 a p para pasar a la siguiente)*

**SOLUCIÓN** (entre paréntesis el número de instrucción a que da solución)**:**

```
var
  v: array[1..3] of integer;
  p_ent: ^integer;
begin
  randomize;
  for i:= 1 to 3 do v[i] := random(100); { (1) }
  for i:= 1 to 3 do write(v[i],' ');  { (2) }
  writeln;
  p_ent := @v[1]; { (3) }
  p_ent^:= 100; { (4) }
  for i:= 1 to 3 do write(v[i],' '); { (5) }
  writeln;
  p_ent^ := 0; { (6) }
  for i:= 1 to 2 do { (6) }
  begin
    p_ent := p_ent + 1;
    p_ent^:= 0;
  end;
  for i:= 1 to 3 do write(v[i],' ');
end
```

3. **Sigue la siguiente secuencia de ejecución escribiendo código Pascal y a la vez dibuja en un papel lo que ocurre tras ejecutar cada instrucción. [Nivel medio]:**

   1. Declara un tipo de registro llamado "nodo" con dos campos: un entero y un puntero a "nodo".
   2. Declara una variable de tipo puntero a nodo

3. Crea un nodo en memoria dinámica (heap)
4. Dale valor 100 al campo entero de este nodo
5. Pon a nil el campo puntero del nodo
6. Pon ahora el puntero del nodo a apuntar al propio nodo
7. En este punto hay dos instrucciones alternativas para liberar la memoria dinámica creada: identifícalas y escríbelas. Dibuja cómo quedarían las cosas tras cada una de ellas.
8. Termina

**SOLUCIÓN** (entre paréntesis el número de instrucción a que da solución)**:**

```
type
  TNodo = record { (1) }
    info: integer;
    sig: ^TNodo;
  end;
var
  pnodo: ^TNodo;    { (2) }
begin
  new(pNodo);
  pNodo^.info := 100; { (3) }
  pNodo^.sig := NIL; { (4) }
  pNodo^.sig := pnodo; { (5) }
  { dispose(pNodo^.sig); // dispose(pnodo); } { (7) Tras ejecutar
cualquiera de las dos pnodo tendría un valor basura y pnodo^.sig
desaparecería pues lo hemos devuelto con dispose y ya no tenemos el
nodo del que formaba parte}
  repeat until keypressed
```